DEI
DEPARTAMENTO
DE ENGENHARIA INFORMÁTICA
TÉCNICO LISBOA

MPI Basics
Collective Communication

Parallel and Distributed Computing

José Monteiro

Friday, 14 March 2025

NOS Award

This year, PDC is being sponsored by NOS.

At the end of the course:

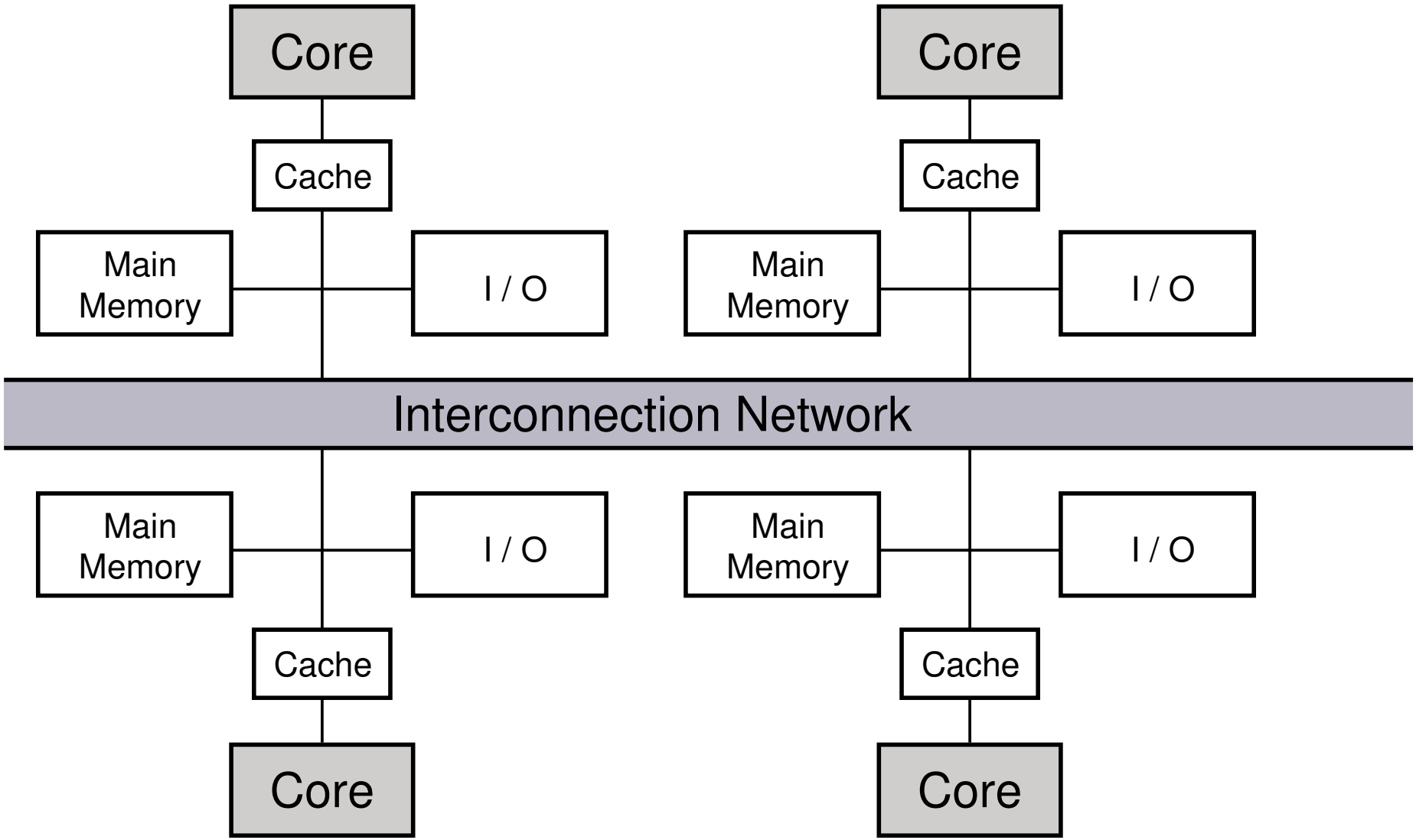
- 3 students with best grades will be selected to give a presentation about their project
- a jury will select the winner taking into account this presentation and the grade

Official information at IST's site.

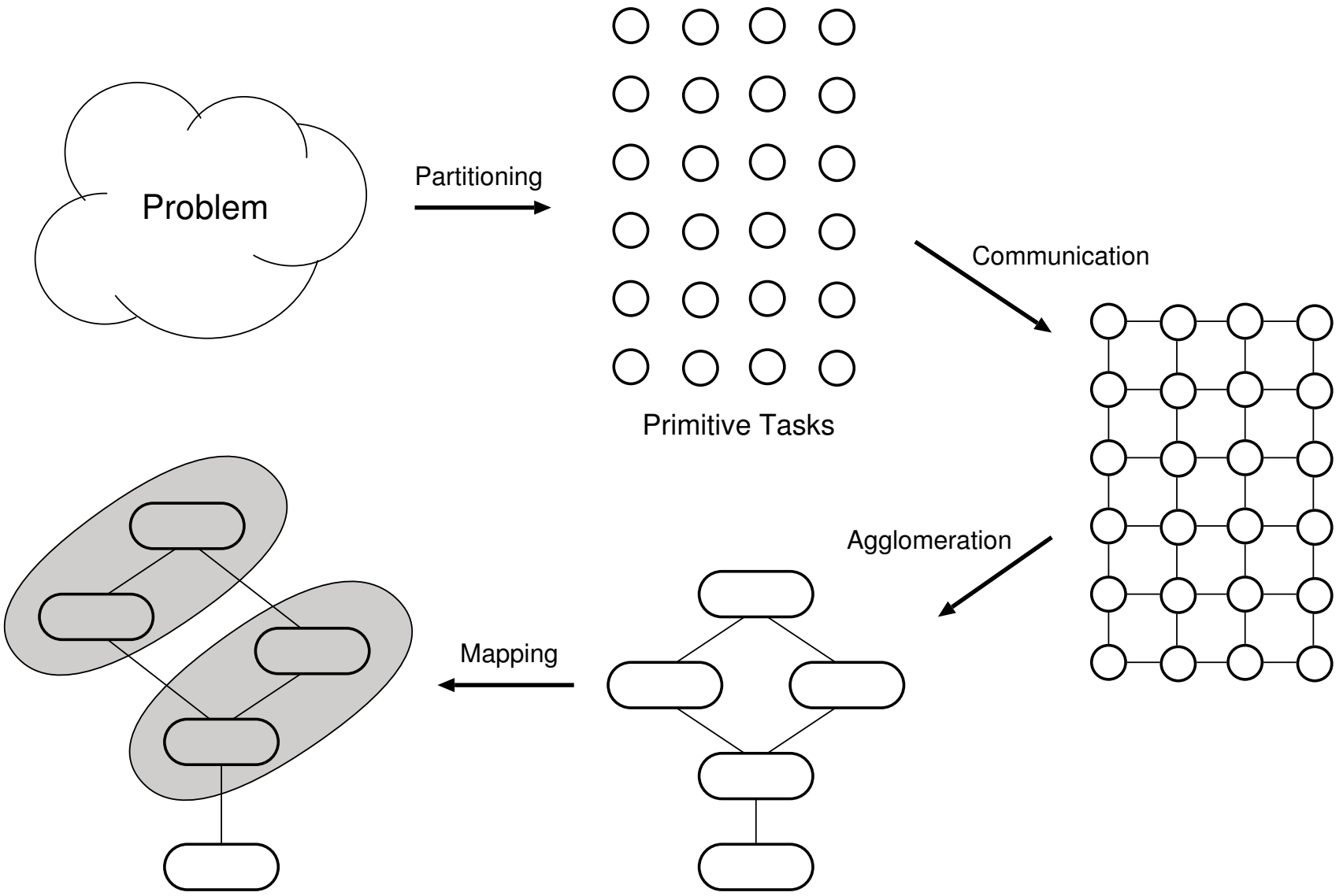
Outline

- MPI
  - Send/Receive: point to point messages
  - Asynchronous messages
  - Messages many-to-many
- Basic MPI application
  - Data decomposition options
  - Parallel algorithm development, analysis
  - Benchmarking
  - Optimizations

Distributed-Memory Systems



Foster's Design Methodology



Message Passing

- Message Passing: programming model for distributed memory parallel computers
  - Every processor executes an independent process
  - Disjoint address spaces, no shared data
  - All communication between processes is done cooperatively, through subroutine calls
- Message passing has succeeded because
  - Maps well to a wide range of hardware
  - Parallelism is explicit and communication is explicit
    - Forces the programmer to tackle parallelization from the beginning
  - MPI makes programs portable

What is MPI?

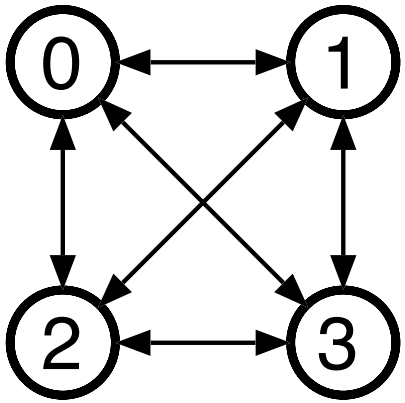
- Message Passing Interface (MPI) is the *de facto* standard for programming on distributed memory parallel computers
  - Library of routines that enable message passing applications
  - Interface specification, not a concrete implementation
- Before MPI
  - Different library for each type of computer
    - CMMD (Thinking Machines CM5)
    - NX (Intel iPSC/860, Paragon)
    - MPL (SP2) and many more …
  - PVM (Parallel Virtual Machine): tried to be a standard, but
    - Not high performance
    - Not carefully specified

MPI History

- MPI was developed by the MPI Forum, a voluntary organization representing industry, government labs and academia
  - MPI-1 (1994): codified existing practice
    - “Who am I?”, “how many processes are there?”
    - Send/recv communication
    - Collective communication e.g. broadcast, reduction, all-to-all
    - Lots of other stuff
  - MPI-2 (1997)
    - Parallel I/O
    - C++/Fortran 90
    - One-sided communication: get/put
  - MPI-3 (2012)
    - Dynamic process creation
    - Asynchronous collectives
    - Persistent collectives
    - Shared-memory windows
  - MPI-4 (2021)
    - Sessions model
    - Function versions (`_c`) with larger sizes
    - Process topologies
    - Improved error handler

An MPI Application

- The elements of the application are
  - 4 tasks, numbered `0` through `3`
  - Communication paths between them



- The set of tasks plus the communication channels is called "`MPI_COMM_WORLD`"

MPI “Hello, World”

```
#include <mpi.h>
#include <stdio.h>

main(int argc, char *argv[])
{
  int me, nprocs;

  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
  MPI_Comm_rank(MPI_COMM_WORLD, &me);

  printf("Hi from node %d of %d\n", me, nprocs);

  MPI_Finalize();
}
```

Compiling and Running

- May be vary between machines, depends on actual MPI implementation
- OpenMPI is a high quality, open-source implementation of MPI
- Merger between well-known MPI implementations
  - FT-MPI, from the University of Tennessee
  - LA-MPI, from Los Alamos National Laboratory
  - LAM/MPI, from Indiana University
  - PACX-MPI, from University of Stuttgart
- Compile:

```
$ mpicc -o hello hello.c
```

- Start four processes:

```
$ mpirun -np 4 ./hello
```

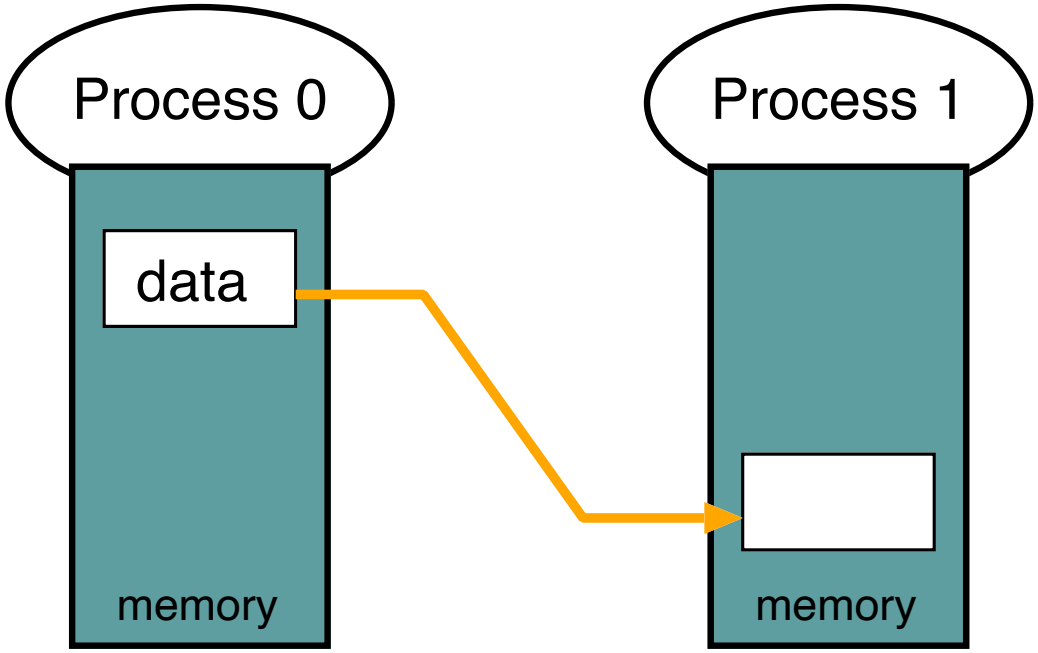
Sample Output

- Run with 4 processes

```
Hi from node 2 of 4
Hi from node 1 of 4
Hi from node 3 of 4
Hi from node 0 of 4
```

- Note that:
  - Order of output may change from run to run
  - Ability to use `stdout` is not guaranteed by MPI

Point-to-Point Communication




```
MPI_Send(data,...)
```

```
MPI_Recv(data,...)
```

Point-to-Point Example

- Task 0 sends array A to task 1, which receives it as B

Task 0

```
#define TAG 123
double A[10];
MPI_Send(A, 10, MPI_DOUBLE, 1, TAG, MPI_COMM_WORLD);
```

Task 1

```
#define TAG 123
double B[10];
MPI_Recv(B, 10, MPI_DOUBLE, 0, TAG, MPI_COMM_WORLD, &status);
```

or

```
MPI_Recv(B, 10, MPI_DOUBLE, MPI_ANY_SOURCE, MPI_ANY_TAG,
         MPI_COMM_WORLD, &status);
```

Source / Destination / Tag

- Send
  - Rank of task message is being sent to (destination)
  - Must be a valid rank (0,⋯ ,N –1) in communicator
- Receive
  - Rank of task message is being received from (source)
  - “Wildcard” `MPI_ANY_SOURCE` matches any source
- Tag
  - On the sending side, specifies a label for a message
  - On the receiving side, must match incoming message
  - On the receiving side, `MPI_ANY_TAG` matches any tag

Using Wildcards

- Unless there is a good reason to do so, do not use wildcards!
- However, there may be good reasons to use wildcards
  - Receiving messages from several sources into the same buffer (use `MPI_ANY_SOURCE`)
  - Receiving several messages from the same source into the same buffer, and don't care about the order (use `MPI_ANY_TAG`)

Pre-Defined Data Types

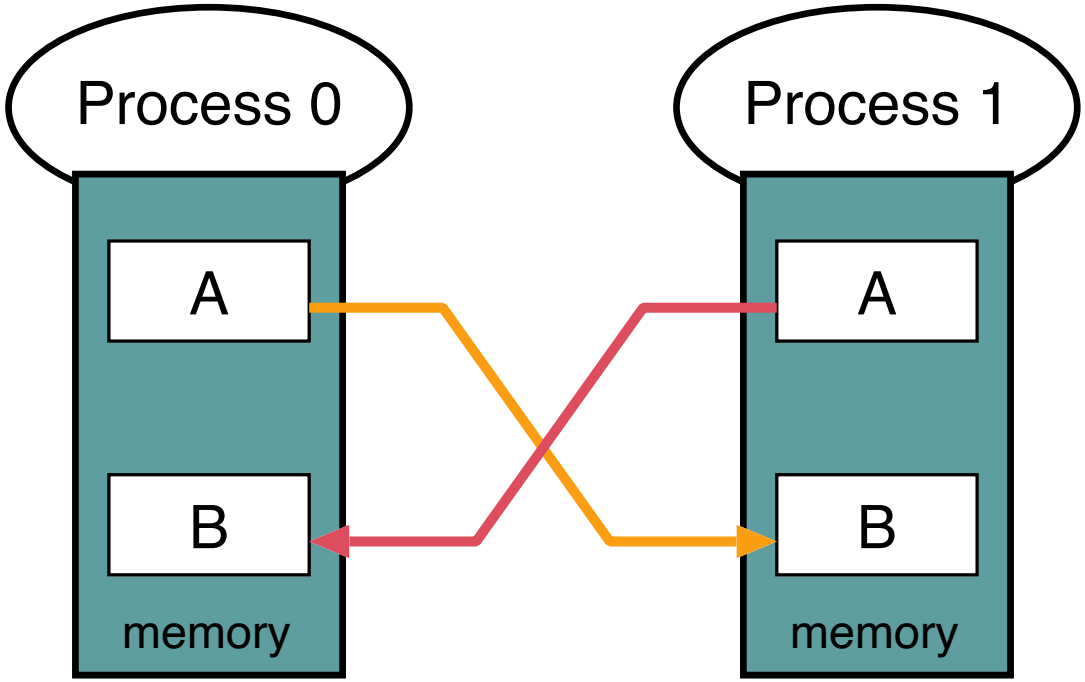
- C
  - `MPI_INT`
  - `MPI_FLOAT`
  - `MPI_DOUBLE`
  - `MPI_CHAR`
  - `MPI_LONG`
  - `MPI_UNSIGNED`
- Type-oblivious
  - `MPI_BYTE`
- MPI allows the definition of other data types/structures

Return Status

- `MPI_Status` is a structure
  - `status.MPI_TAG` is tag of incoming message (useful if `MPI_ANY_TAG` was specified)
  - `status.MPI_SOURCE` is rank of the sender of incoming message (useful if `MPI_ANY_SOURCE` was specified)
  - How many elements of given datatype were received

```
MPI_Get_count(MPI_Status *status,
              MPI_Datatype datatype,
              int *count)
```

Swapping Data




```
MPI_Send(A,...)
MPI_Recv(B,...)
```

```
MPI_Send(A,...)
MPI_Recv(B,...)
```

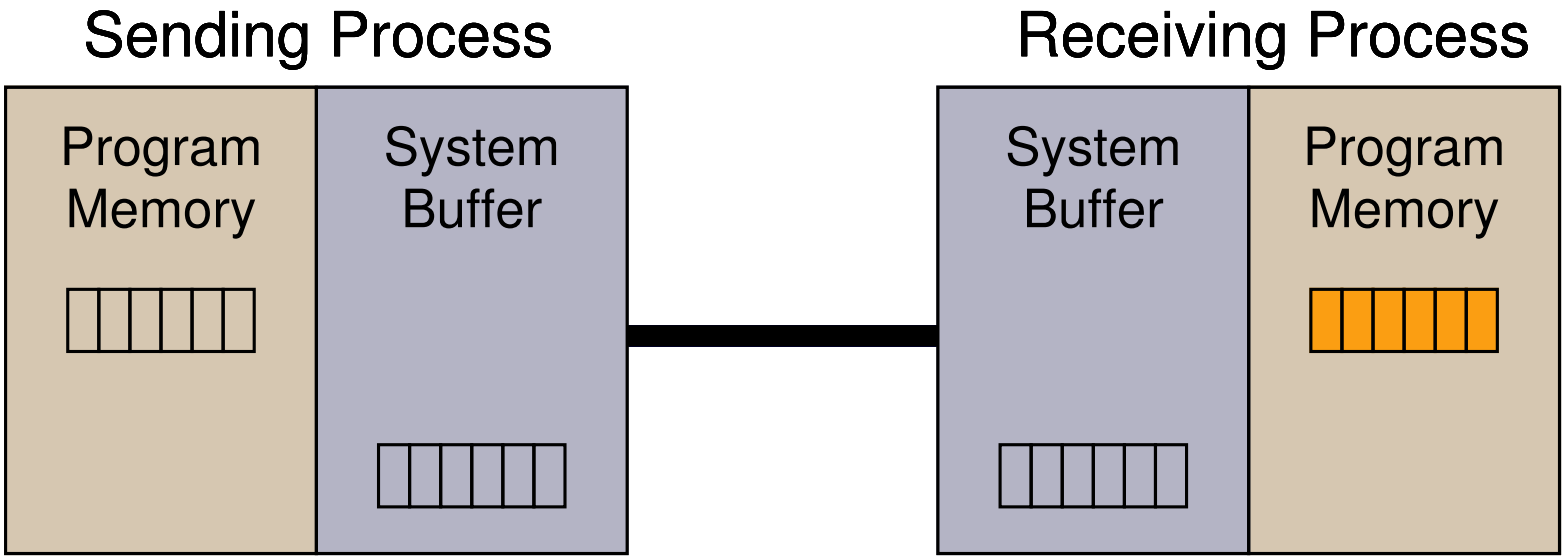
Blocking Operations

- `MPI_Send()` is blocking
  - Only returns when the send buffer can be reused
    - Data has been received by destination
    - Or MPI has saved the data somewhere

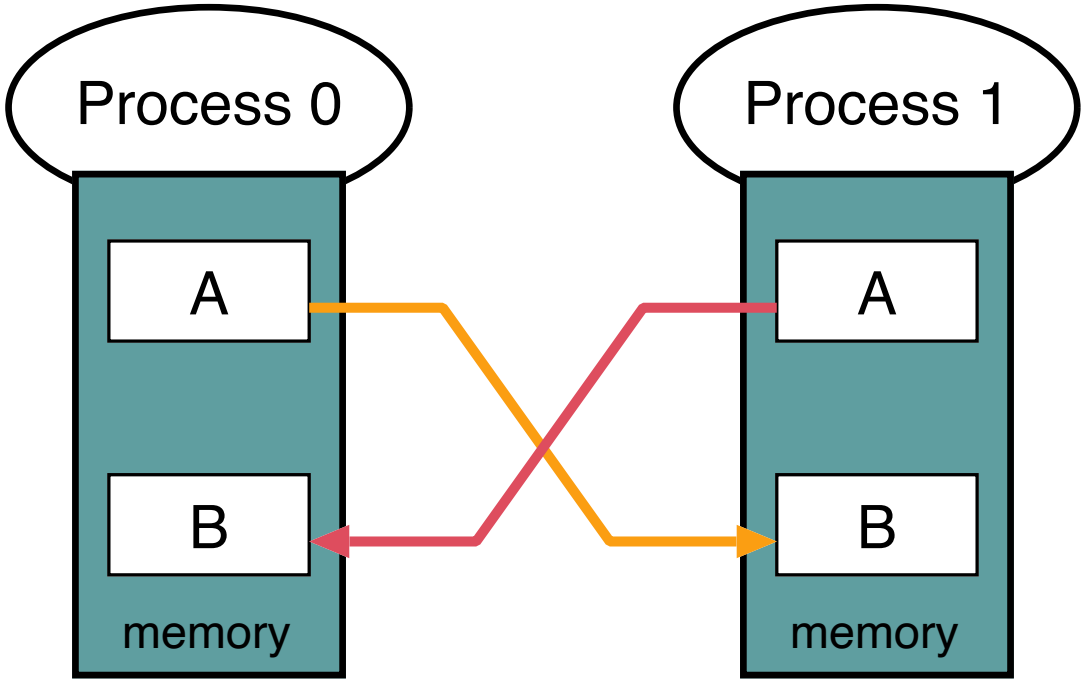
- `MPI_Recv()` is blocking
  - Only returns when the receive buffer has been filled with valid data

Does this work?

Internals of Send and Receive



Swapping Data




```
MPI_Send(A,...)
MPI_Recv(B,...)
```

```
MPI_Send(A,...)
MPI_Recv(B,...)
```

- Requires buffering to avoid deadlock!

  → a correct program does not rely on buffering!

Non-blocking Operations

```
#define MYTAG 123
#define WORLD MPI_COMM_WORLD
MPI_Request request;
MPI_Status status;
```

Task 0

```
MPI_Irecv(B, 100, MPI_DOUBLE, 1, MYTAG, WORLD, &request);
MPI_Send(A, 100, MPI_DOUBLE, 1, MYTAG, WORLD);
MPI_Wait(&request, &status);
```

Task 1

```
MPI_Irecv(B, 100, MPI_DOUBLE, 0, MYTAG, WORLD, &request);
MPI_Send(A, 100, MPI_DOUBLE, 0, MYTAG, WORLD);
MPI_Wait(&request, &status);
```

- No deadlock
- Data may be transferred concurrently

Overlapping Communication and Computation

- On some computers it may be possible to do useful work while data is being transferred: latency reduction

```
MPI_Request requests[2];
MPI_Status statuses[2];

MPI_Irecv(B, 100, MPI_DOUBLE, p, 0, WORLD, &request[1]);
MPI_Isend(A, 100, MPI_DOUBLE, p, 0, WORLD, &request[0]);

 .... do some useful work here ....

MPI_Waitall(2, requests, statuses);
```

  - `Irecv`/`Isend` initiate communication
  - Communication proceeds while processor is doing useful work
  - Hardware support necessary for true overlap

Operations on `MPI_Request`

- `MPI_Wait(INOUT request, OUT status)`
  - Waits for operation to complete
  - Returns information in `status`
  - Frees request object (and sets to `MPI_REQUEST_NULL`)
- `MPI_Test(INOUT request, OUT flag, OUT status)`
  - Tests to see if operation is complete
  - Returns information in `status` if complete
  - Frees request object if complete
- `MPI_Cancel(IN request)`
  - Cancels and completes a request
- `MPI_Waitall(..., INOUT array of requests, ...)`
- `MPI_Testall(..., INOUT array of requests, ...)`
- `MPI_Waitany/MPI_Testany/MPI_Waitsome/MPI_Testsome`

Non-blocking Communication Issues

- Obvious concerns
  - May not modify the buffer between **`Isend()`** and the corresponding **`Wait()`**
    - Results are undefined
  - May not look at or modify the buffer between **`Irecv()`** and the corresponding **`Wait()`**
    - Results are undefined
  - May not have two pending **`Irecv()`** for the same buffer
- Less obvious
  - May not *look* at the buffer between **`Isend()`** and the corresponding **`Wait()`**
  - May not have two pending **`Isend()`** for the same buffer

Information about Received Message

- It is possible to check information about an expected message before actually receiving it:

```
MPI_Probe(IN source, IN tag, OUT status)
```

  - Blocks until message is received
  - Returns information in `status`
  - Message not yet received, still requires `MPI_Recv`

- Non-blocking version:

```
MPI_Iprobe(IN source, IN tag, OUT flag, OUT status)
```

Communicators

- A communicator is an object that represents
  - A set of tasks
  - Private communication channels between those tasks

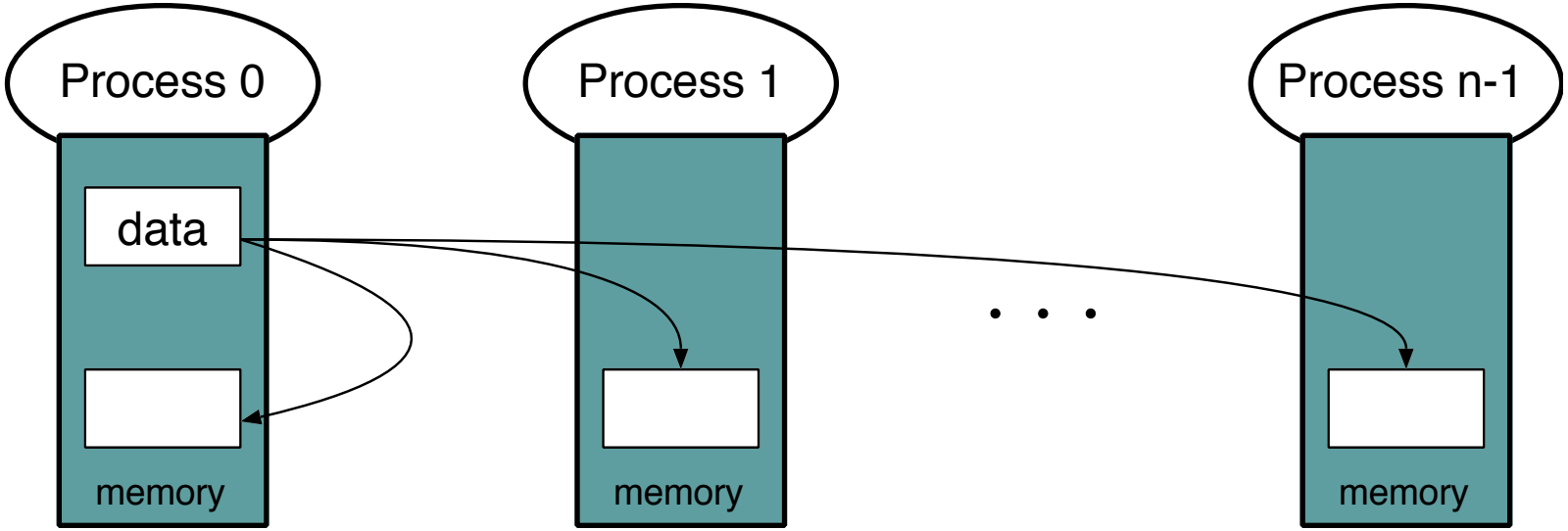
- `MPI_COMM_WORLD` is a communicator that includes all tasks and that is available at startup

- Communicators allow for the definition of the scope for collective operations

Collective Operations

- Collective communication is communication among a group of tasks
  - Broadcast
  - Scatter/gather
  - Global operations (reductions)
  - Parallel prefix (scan)
  - Synchronization (barrier)

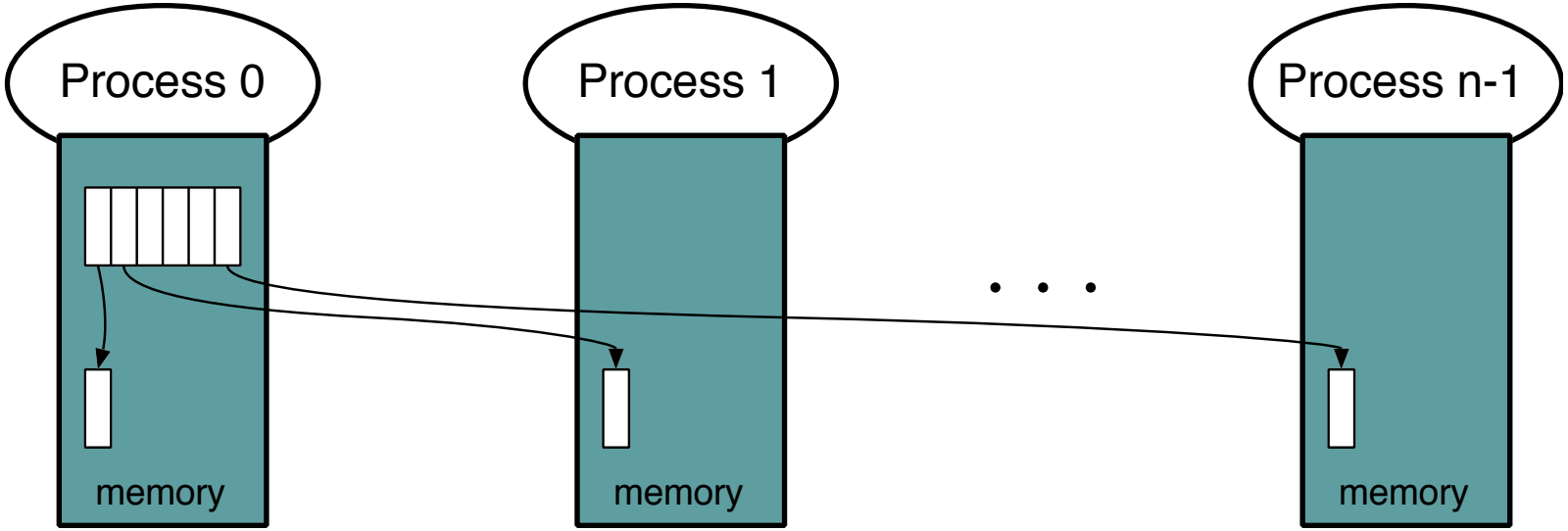
Broadcast




```
MPI_Bcast(buf, len, type, root, comm)
```

- Task with rank = `root` is source of data (in `buf`)
- Other tasks receive data

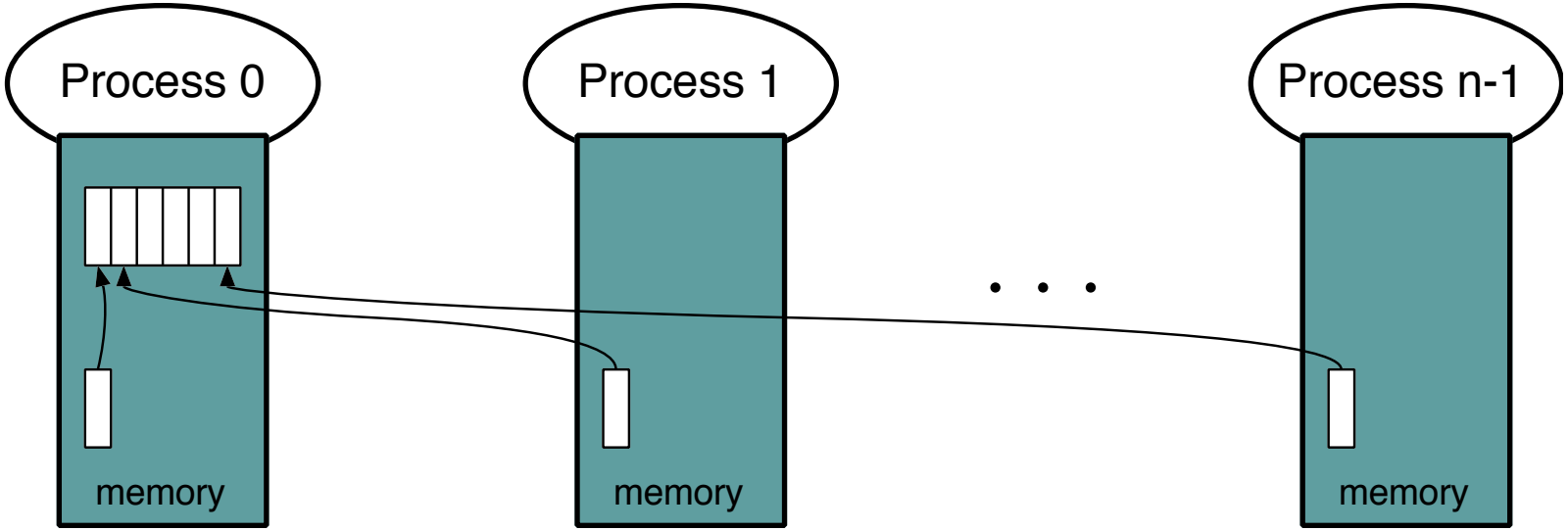
Scatter



`MPI_Scatter()` / `MPI_Scatterv()`

– Subparts of a single large array are distributed to tasks

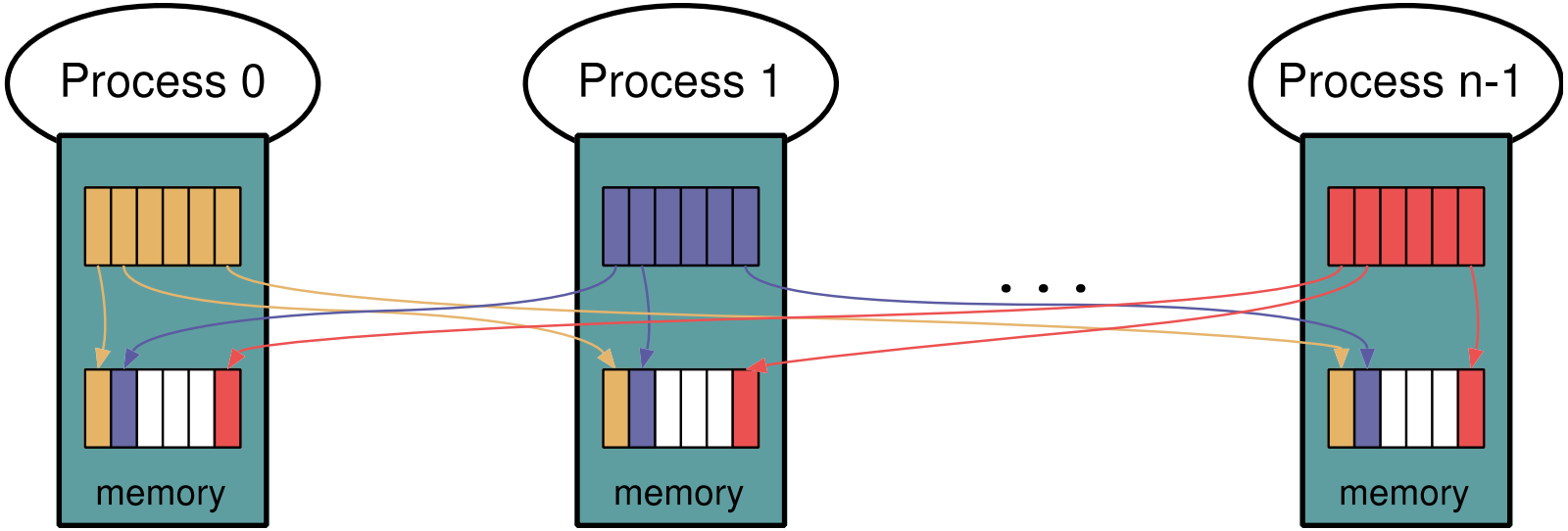
Gather




```
MPI_Gather() / MPI_Gatherv()
MPI_Allgather() / MPI_Allgatherv()
```

– Each task contributes local data that is gathered into a larger array

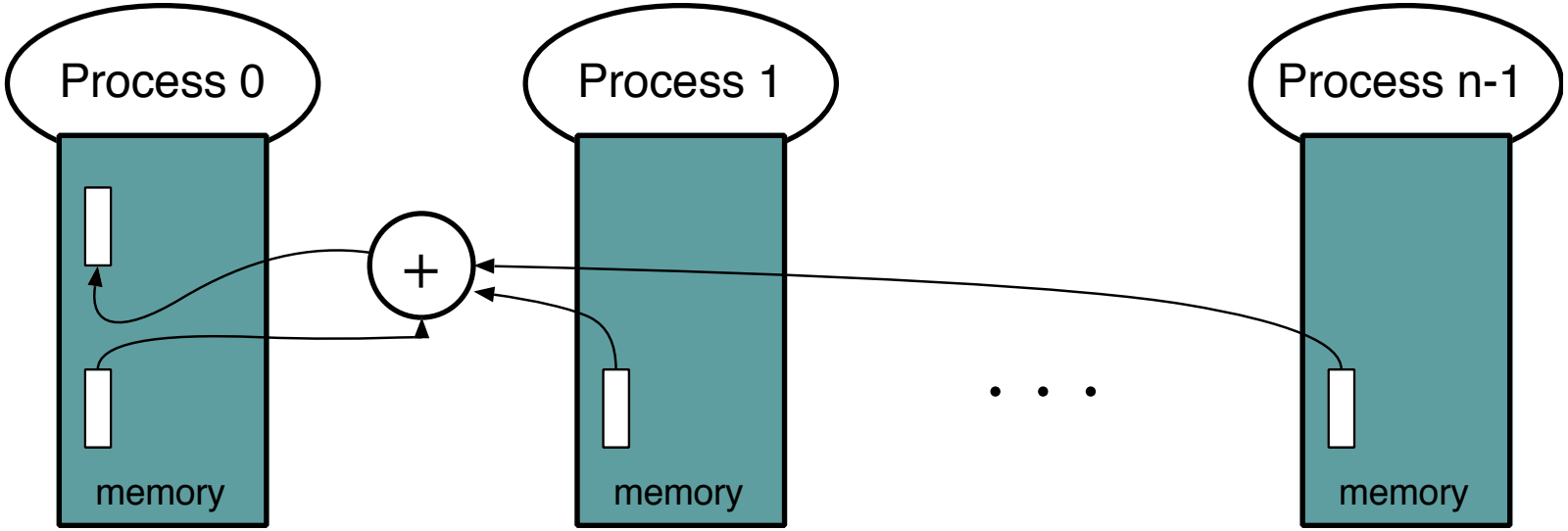
All to All Data Movements



`MPI_Alltoall(sndbf, sndct, sndt, rcvbf, rcvct, rcvt, comm)`

- All tasks send and receive data from all other tasks
- For a communicator with N tasks
  - `sndbf` contains P blocks of `sndct` elements each
  - `rcvbf` receives P blocks of `rcvct` elements each
  - each task sends block **`i`** of `sndbf` to task **`i`**
  - each task receives block **`j`** of `rcvbf` from task **`j`**

Reduction




```
MPI_Reduce(indata, outdata, count, type, op, root, comm)
MPI_Allreduce(indata, outdata, count, type, op, comm)
```

– Combine elements in input buffer from each task, placing result in output buffer.

Reduction

- Reduce: output appears only in buffer on root
- Allreduce: output appears on all processes

- Some operation types
  - `MPI_SUM`
  - `MPI_PROD`
  - `MPI_MAX`
  - `MPI_MIN`
  - `MPI_BAND`

- Arbitrary user-defined operations on arbitrary user-defined datatypes are possible

Reduction: Dot Product Example

```
/* distribute two vectors over all processes such that
   processor 0 has elements 0...99
   processor 1 has elements 100...199
   processor 2 has elements 200...299 etc. */

double dotprod(double a[100], double b[100])
{
  double gresult = lresult = 0.0;
  int i; /* compute local dot product */
  for(i = 0; i < 100; i++)
      lresult += a[i] * b[i];

  MPI_Allreduce(&lresult, &gresult, 1, MPI_DOUBLE, MPI_SUM,
                MPI_COMM_WORLD);

  return(gresult);
}
```

Synchronization

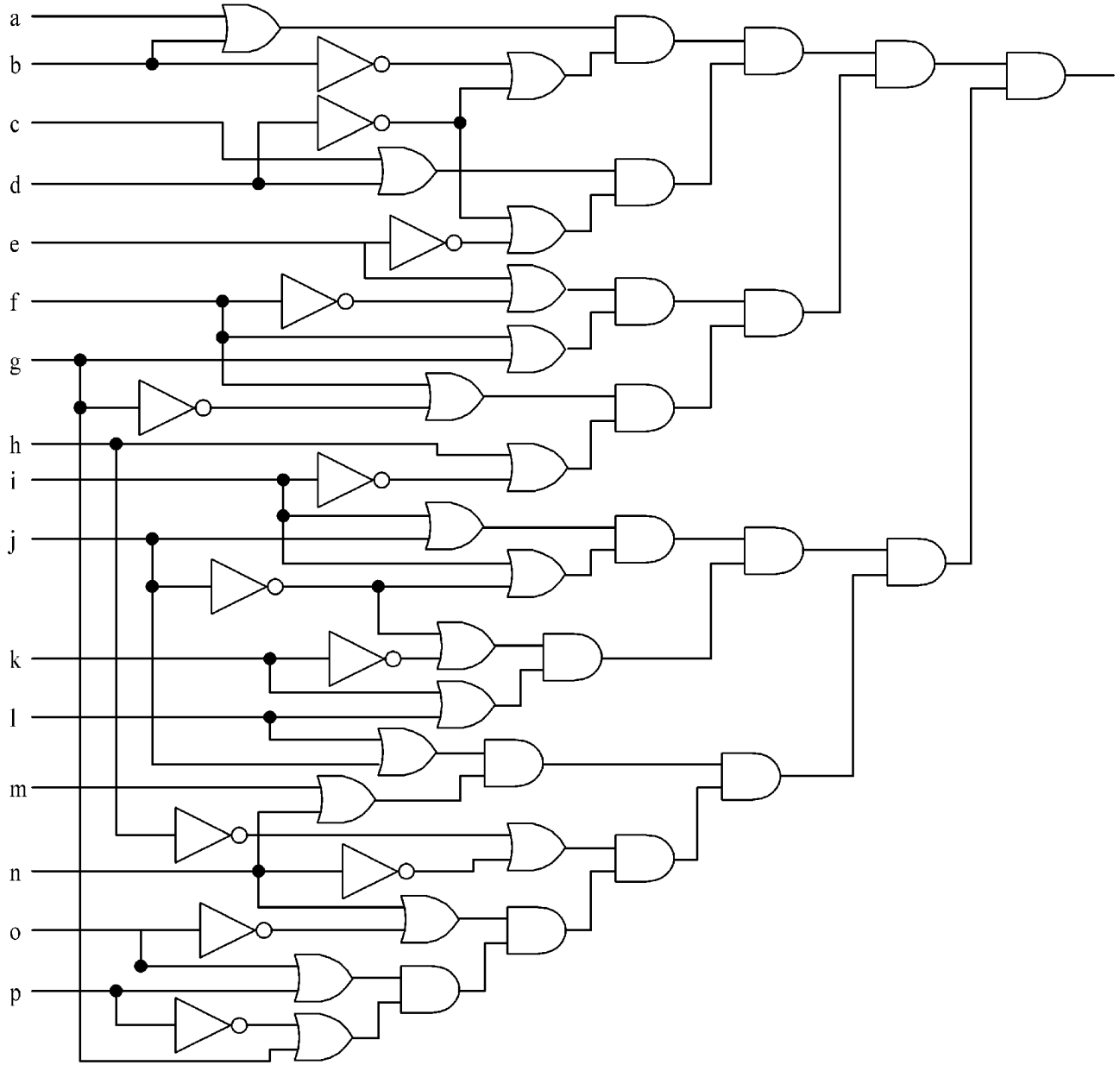
```
MPI_Barrier(communicator)
```

– No process leaves the barrier until all processes have entered it

First MPI Problem: Circuit Satisfiability

- Circuit Satisfiability (CSAT)
  - Determine a combination of input values that assert the output node of a logic circuit, or prove that no such combination is possible
  - Basic problem in VLSI circuit testing
  - NP-Complete problem!

CSAT Instance



CSAT Problem

- Determine all input combinations that assert the circuit output
  - Assert = make it assume logic value `1` (true)

- We will solve this problem through exhaustive search
  - Test all input combinations!

Foster's Methodology on CSAT

- Partitioning
  - Make each input combination test a primitive task
- Communication
  - No channels!
    - No interaction between tasks
- Aggregation and mapping
  - Distribute tasks evenly among available CPUs

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  for (i = id; i < 65536; i += p)
    check_circuit (id, i);

  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  return 0;
}
```

C Program with MPI

```
/* Return 1 if 'i'th bit of 'n' is 1; 0 otherwise */
#define EXTRACT_BIT(n,i) ((n&(1<<i))?1:0)

void check_circuit(int id, int z) {
  int i, v[16]; /* Each element is a bit of z */
  for (i = 0; i < 16; i++)  v[i] = EXTRACT_BIT(z,i);
  if ((v[0] || v[1]) && (!v[1] || !v[3]) && (v[2] || v[3])
    && (!v[3] || !v[4]) && (v[4] || !v[5]) && (v[5] || !v[6])
    && (v[5] || v[6]) && (v[6] || !v[15]) && (v[7] || !v[8])
    && (!v[7] || !v[13]) && (v[8] || v[9]) && (v[8] || !v[9])
    && (!v[9] || !v[10]) && (v[9] || v[11]) && (v[10] || v[11])
    && (v[12] || v[13]) && (v[13] || !v[14]) && (v[14] || v[15])){
    printf("%d) %d%d%d%d%d%d%d%d%d%d%d%d%d%d%d%d\n", id,
      v[0],v[1],v[2],v[3],v[4],v[5],v[6],v[7],v[8],v[9],
      v[10],v[11],v[12],v[13],v[14],v[15]);
    fflush(stdout);
  }
}
```

Compiling and Running

- Compile command

```
$ mpicc <flags> -o <executable> <source>.c
```

  - Same flags as **gcc**, and more
  - Links with MPI libraries

- In our case

```
$ mpicc -O2 -o csat1 csat1.c
```

- Executing

```
$ mpirun -np <p> <executable> <args>
```

  - `<args>`: program's command-line arguments

Sample Output: 1 Task

```
$ mpirun -np 1 csat1
0) 1010111110011001
0) 0110111110011001
0) 1110111110011001
0) 1010111111011001
0) 0110111111011001
0) 1110111111011001
0) 1010111110111001
0) 0110111110111001
0) 1110111110111001
Process 0 is done
```

Sample Output: 2 Tasks

```
$ mpirun -np 2 csat1
0) 0110111110011001
0) 0110111111011001
0) 0110111110111001
1) 1010111110011001
1) 1110111110011001
1) 1010111111011001
1) 1110111111011001
1) 1010111110111001
1) 1110111110111001
Process 0 is done
Process 1 is done
```

Sample Output: 3 Tasks

```
$ mpirun -np 3 csat1
0) 0110111110011001
0) 1110111111011001
2) 1010111110011001
1) 1110111110011001
1) 1010111111011001
1) 0110111110111001
0) 1010111110111001
2) 0110111111011001
2) 1110111110111001
Process 1 is done
Process 2 is done
Process 0 is done
```

Deciphering the Output

- Output order only partially reflects order of output events inside parallel computer
  - If process A prints two messages, first message will appear before second
  - If process A calls **`printf`** before process B, there is no guarantee process A's message will appear before process B's message

Collective Communication

- Enhancing the program:

  write a new version of the Circuit Satisfiability program so that it returns the total number of solutions

- Modify function **`check_circuit`**
  - Return `1` if circuit satisfiable with input combination
  - Return `0` otherwise

- Each task keeps a local count of satisfiable input combinations it has found

- Incorporate sum-reduction into program

New Declarations and Code

```
int count;          /* Local sum */
int global_count; /* Global sum */
int check_circuit (int, int);

...

count = 0;
for (i = id; i < 65536; i += p)
  count += check_circuit (id, i)
```

Prototype of MPI Reduce

```
int MPI_Reduce (
  void          *operand, /* addr of 1st reduction element */
  void          *result,  /* addr of 1st reduction result */
  int           count,    /* reductions to perform */
  MPI_Datatype  type,     /* type of elements */
  MPI_Op        operator, /* reduction operator */
  int           root,     /* process getting result(s) */
  MPI_Comm      comm      /* communicator */
)
```

```
MPI_Reduce (&count, &global_count, 1, MPI_INT, MPI_SUM, 0,
            MPI_COMM_WORLD);
```

Only this task gets the result!

New Version of C Program

```
#include <mpi.h>
#include <stdio.h>

int main (int argc, char *argv[])
{
  int i, id, p;
  int count, global_count;

  MPI_Init (&argc, &argv);
  MPI_Comm_rank (MPI_COMM_WORLD, &id);
  MPI_Comm_size (MPI_COMM_WORLD, &p);

  count = 0;
  for (i = id; i < 65536; i += p)
    count += check_circuit (id, i);
  MPI_Reduce (&count, &global_count, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
  printf ("Process %d is done\n", id);
  fflush (stdout);
  MPI_Finalize ();
  if (!id) printf("There are %d different solutions\n", global_count);
  return 0;
}
```

Sample Output of New Version: 3 Tasks

```
$ mpirun -np 3 csat2
0) 0110111110011001
0) 1110111111011001
1) 1110111110011001
1) 1010111111011001
2) 1010111110011001
2) 0110111111011001
2) 1110111110111001
1) 0110111110111001
0) 1010111110111001
Process 1 is done
Process 2 is done
Process 0 is done
There are 9 different solutions
```

Benchmarking Code

- Metric of interest: real time!

```
double MPI_Wtime()
```

Time in seconds since an arbitrary time in the past

```
double MPI_Wtick()
```

Timer resolution

- How to eliminate startup times?

```
int MPI_Barrier(MPI_Comm comm)
```

Barrier synchronization

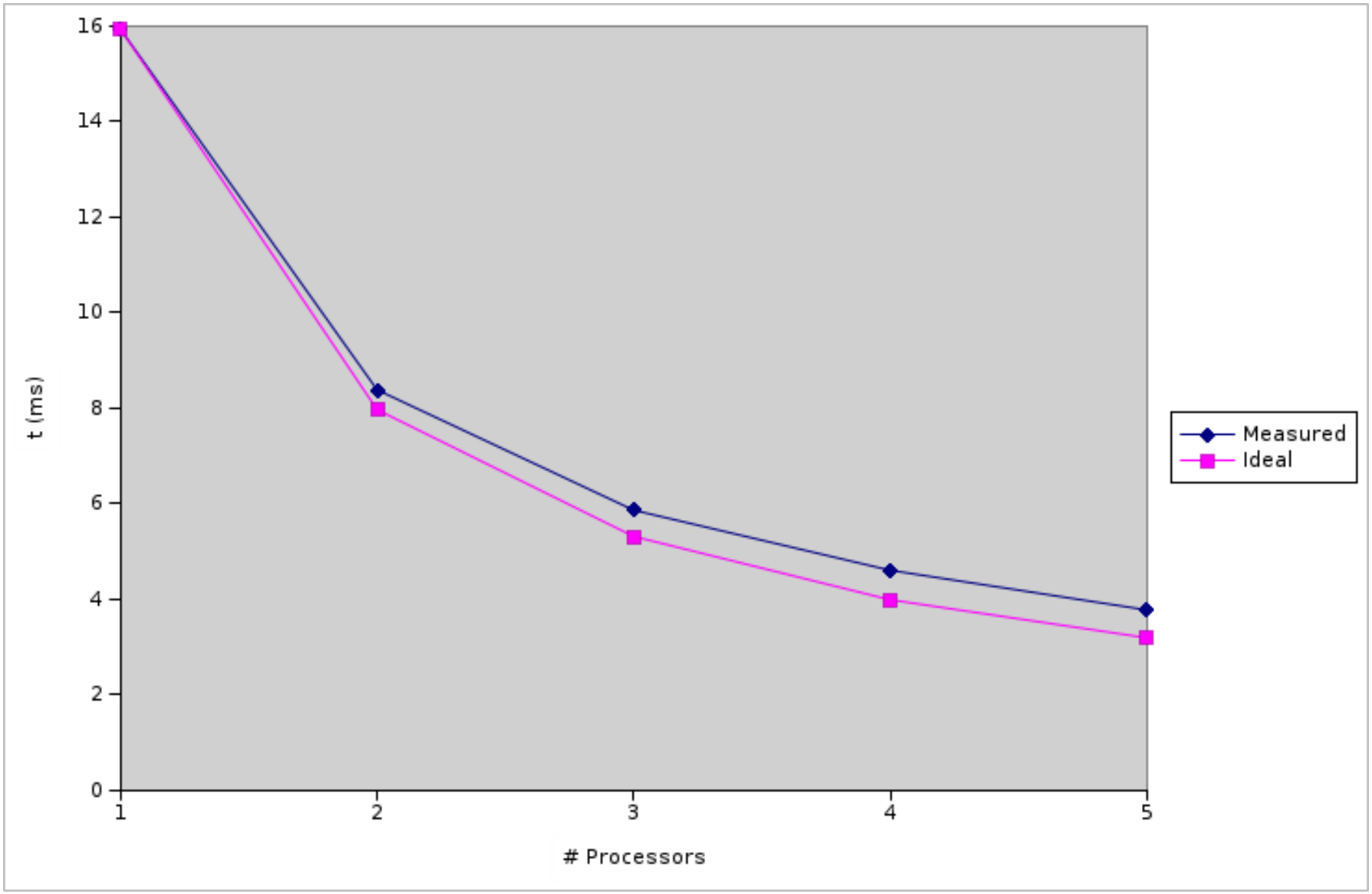
Benchmarking Code

```
double elapsed_time;

...

MPI_Init (&argc, &argv);
MPI_Barrier (MPI_COMM_WORLD);
elapsed_time = - MPI_Wtime();

...

MPI_Reduce (...);
elapsed_time += MPI_Wtime();
```

Benchmarking Results

Processors	Time (ms)
1	15,93
2	8,38
3	5,86
4	4,60
5	3,77

Benchmarking Results



Review

- MPI
  - Send/Receive: point to point messages
  - Asynchronous Messages
  - Messages Many-to-many

- Basic MPI application
  - Data decomposition options
  - Parallel algorithm development, analysis
  - Benchmarking
  - Optimizations

Next Class …

- More complex application examples with MPI